Chapter 3

- in this chapter the agent discovers which action it should take , when the environment is not clear enough it shall find a way
- **informed algorithms** : the agent can estimate how far it is from the goal.
- **uninformed algorithms** : we can't estimate how far we are from the goal .
  In thsi chapter we will assume that the agent always has access to information abou the world (he has the map to Romania)

Problem Solving Agent

Goal formulation:

The agent adopts the goal of reaching *Bucharest*. Goals organize Goal formulation behavior by limiting the objectives and hence the actions to be considered.

Problem Formulation

Description of the states and actions necexssary to reach the goal

Solution :

- a sequence of actions that the agent executes that leads to him reaching the goal .

Execution :

- The agent can now execute the actions in the solution, one at a time.

Closed and Open loop systems :

- **open loop** : once the agnet has found a solution to the problem it ignores all the other percepts and only focuses on executing the sequence of the solution it has found "*It relies entirely on the correctness of its model of the environment.* "
- **Closed Loop** : the agent continuously monitors the environment through percepts (sensory input) and adjusts its actions based on feedback. This approach is more robust in uncertain or dynamic environments.
  - *Example* : driving to Bucharest but encounters a roadblock in Fagaras, it can detect this change (via percepts) and replan its route accordingly.

Search Problems and Solutions :

Search Problem

- we can define a **search problem** as follows :

1. **State Space** :
   - The set of All Possible states that the enviornment can be in (in Arad , Bucharest..)
2. **Initial State** :
   - The state where the agent startes its search .

3. **Goal States** :
   - the state(s) the agent aims to reach , there can be one / multiple/ defined by a property .
   - *Example* : Bucharest , any state where no dirt is present .
4. **Actions** :
   - The set of All Actions available to the agent in given state (these actions are aplicabale in that state)
   - *Example* : `ACTIONS(Arad) = {ToSibiu, ToTimisoara, ToZerind}`
5. **Transition Model** :
   - Describes the result of taking an action in a state. It maps a state and an action to a new state.
   - *Example* : `RESULT(Arad, ToZerind) = Zerind`
6. **Action Cost Function** :
   - assigns a numeric cost to each action , the cost reflects the performance measure of the agnet (distance , time , energy...)
   - *Example* : from Arad to Zerind it costs 75km

Path :

- A sequence of actions .

Solution :

- path from the initial state to a goal state .

Optimal Solution :

- A solution (path) with the lowest total cost among all possible solutions

State Space represnetation:

- we can represent a state space in a graph where :
  - **Vertices (Nodes)**: Represent states (e.g., cities)
  - **Edges**: Represent actions (e.g., roads between cities).

> **Remarks**
>
> 1. **Action Costs are Additive**: The total cost of a path is the sum of the individual action costs.
> 2. **Positive Action Costs**: All action costs are positive to avoid complications like infinite loops in negative-cost cycles.
> 3. **Finite Actions**: The number of actions in each state is finite (though the text mentions techniques for infinite actions).

Formulating Problems

- Process of defining a problem in a way that it can be solved computationally . (Thsi involves identifying the relavent components of the problem such as the *states* , *actions* and *goals*)

Abstraction :

- removing unnecessary details from a problem to focus on the essential aspects.

Approproiate levels of abstraction :

- **Too much Details** : makes the problem unwiedly
- **Too Little Details** : Risks missing important aspects.
- **Right Level** : Balances simplicity and relevance.

Validity of Abstraction :

- An abstraction is valid if every abstract solution can be expanded into a real-world solution.

Usefulness of Abstraction :

- Abstract actions should be easier to execute than solving the original problem .

2. Example Problems

Standardized Problems :

- provide a controlled environment for testing and comparing problem-solving algorithms.
- *Examples* : A **grid world** where an agent (like a robot) moves around a grid, avoiding obstacles and picking up objects.

Real-World Problems :

- They are practical everyday problems , more complex and require tailored solutions due to their unique constraints and environments.
- *Example* : A **robot navigating** a warehouse, where its sensors and the layout of the warehouse are unique to that problem
  (examples to be added later)

3. Search Algorithms

- **Search Algorithm** takes as input the search problem and returns a solution or an indication of failure .
  - they explore possible paths from the starting point (intitial state) to the goal by building a *search tree*

3.1 Search Tree :

- A way to represent the exploration of possible paths in the state space
  - **Nodes** : each node in the tree represents a state in the problem .
  - **Edges** : the edges represent actions that move the agent from one state to another .
  - **Root** : is the intitial state .

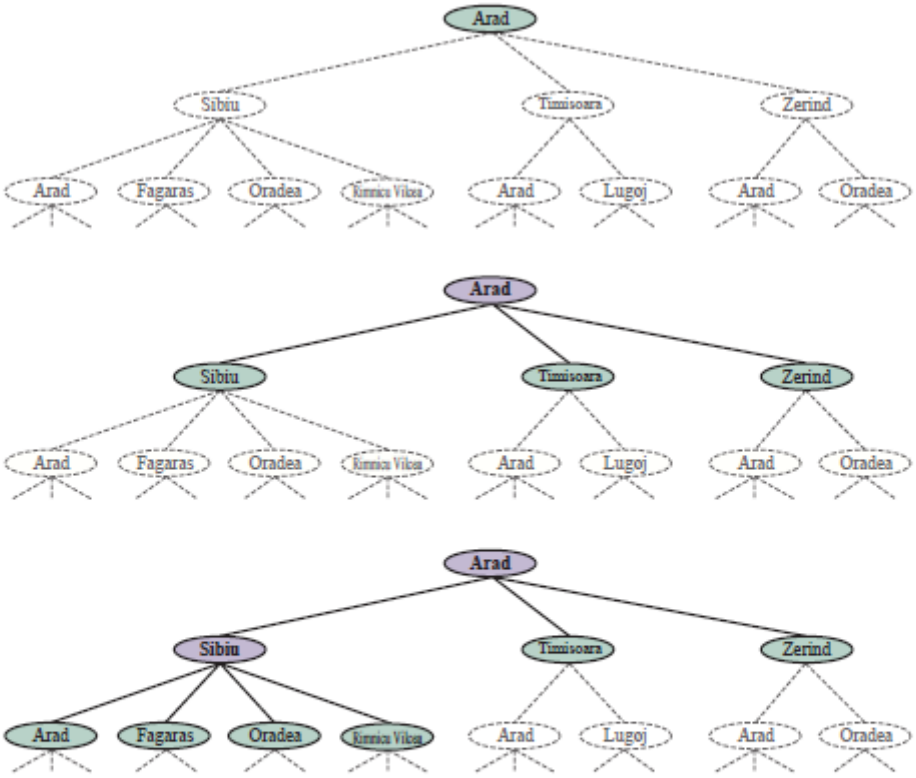
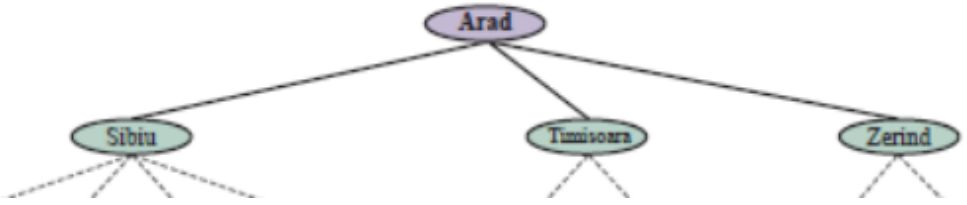


Figure 3.4 Three partial search trees for finding a route from Arad to Bucharest. Nodes that have been *expanded* are lavender with bold letters; nodes on the frontier that have been *generated* but not yet expanded are in green; the set of states corresponding to these two types of nodes are said to have been *reached*. Nodes that could be generated next are shown in faint dashed lines. Notice in the bottom tree there is a cycle from Arad to Sibiu to Arad; that can't be an optimal path, so search should not continue from there.

- During the search, at each node (state), the current state is expanded (new set of states is generated by applying each legal action to the current state.)
  Key Decision :
- The Algorithm must decide which child node to expand next
  - *Example* : From Arad, should the algorithm explore Sibiu, Timisoara, or Zerind first?
- The algorithm can choose one node to expand but keep the others in reserve (in case the chosen path doesn't lead to the goal).
  Search Strategy :
- The method the algorithm uses to decide which node to expand next (Breadth first search , depth first search ...)
  Frontier :
- The frontier is the set of all leaf nodes (nodes that haven't been expanded yet) in the search tree.
- The algorithm expands nodes from the frontier until it finds a solution or runs out of nodes to expand.
- ones in green !



Loopy / Redundant Paths :

- **Loopy** :(a special case of redundant paths) a path that revisits the same state multiple times , creating a loop (Arad → Sibui → Arad → Sibui ....)
- **Redundant** : multiple paths lead to the same state
  - *Example* : in the 8 Queens Problem , we can avoid redundant paths by keeping unqiue postitions for the queens , but sometimes it is unavoidable where actions are *reversible* .
  Explored Set :
- Tree Search Algorithm can be augmented with a data structure called the *explored set* , where we keep all the visited (expanded nodes) so as not to repeat expanding it .

> **How**
>
> 1. **Before adding a Node to The Frontier :**
> - The algorithm checks if the state is already in the **explored set** or the **frontier**.
> - If it is, the node is discarded (to avoid redundancy).
> - If it isn't, the node is added to the frontier.

3.1.2 Tree Search

```
function TREE-SEARCH(problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        expand the chosen node, adding the resulting nodes to the frontier
```

3.1.3 Graph Search

```
function GRAPH-SEARCH(problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    initialize the explored set to be empty
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        add the node to the explored set
        expand the chosen node, adding the resulting nodes to the frontier
            only if not in the frontier or explored set
```

- *Example* : Route finding on a grid

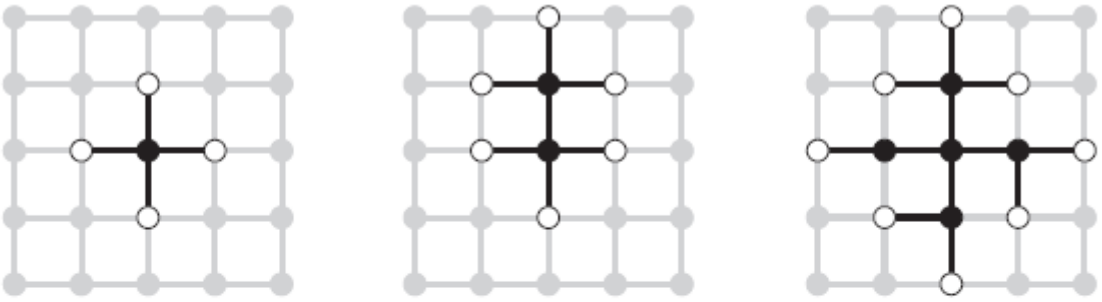


Figure 3.9 The separation property of GRAPH-SEARCH, illustrated on a rectangular-grid problem. The frontier (white nodes) always separates the explored region of the state space (black nodes) from the unexplored region (gray nodes). In (a), just the root has been expanded. In (b), one leaf node has been expanded. In (c), the remaining successors of the root have been expanded in clockwise order.

3.2 Infrastructure for Search Algorithms

3.2.1 Node Structure in the Search Tree

1. **n.STATE** : The state in the state space that the node represents (a City in romania Problem)
2. **n.PARENT** The node that generated this node
3. **n.ACTION** : the Action taken to move from the parent node to the current node
4. **n.PATH-COST** : The goal cost of the path from the initial state to the current node $g(n)$ - *Example* : If the cost from Arad to Sibiu is 140 km, and from Sibiu to Bucharest is 99 km, the path cost for Bucharest woul be 140+99=239140+99=239 km.

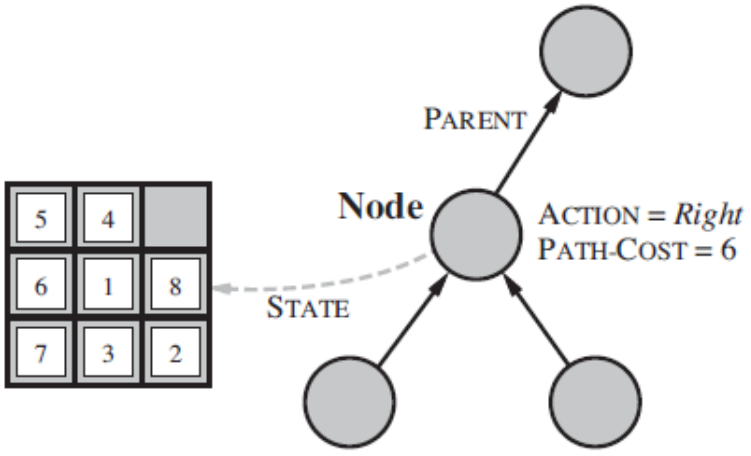


Figure 3.10 Nodes are the data structures from which the search tree is constructed. Each has a parent, a state, and various bookkeeping fields. Arrows point from child to parent.

The function **CHILD-NODE** takes a parent node and an action and returns the resulting child node:

```
function CHILD-NODE(problem, parent, action) returns a node
  return a node with
      STATE = problem.RESULT(parent.STATE, action),
      PARENT = parent, ACTION = action,
      PATH-COST = parent.PATH-COST + problem.STEP-COST(parent.STATE, action)
```

3.2.2 Frontier Data Structure Queues :

- The frontier which holds the leaf nodes to be expanded can be stored using a queue and the method in which we pop the Elements from it can be :

1. **FIFO Queue (First-In, First-Out)**:
   - The oldest node (first added) is popped first.
   - Used in **Breadth-First Search (BFS)**.
2. **LIFO Queue (Last-In, First-Out)**:
   - The newest node (last added) is popped first.
   - Also known as a **stack**, used in **Depth-First Search (DFS)**.
3. **Priority Queue**:
   - Nodes are popped based on a priority value (e.g., lowest cost or highest heuristic value).
   - Used in **Uniform Cost Search (UCS)** and **A Search**

3.2.2 Explored Set Data Structure Hash Tables:

- The explored set can be implemented using the hash table for *efficient lookup* and *insertion* .

  - **Equality of States**: The hash table must correctly identify when two states are the same, even if they are represented differently.
    - Example: In the Traveling Salesperson Problem (TSP), the set of visited cities {Bucharest, Urziceni, Vaslui} should be treated as the same as {Urziceni, Vaslui, Bucharest}.
    - **Canonical Form**: To ensure equality, states should be represented in a standardized way (e.g., sorted lists or bit-vectors).

  (sorted to fix the order issue , bit vector for encoding ogicially equivalent states the same way)

3.2 Measuring Problem-Solving

1. **Completness** :
   - An Algorithm is complete if it is guaranted to find a solution when one exists . (BFS -> Complete , DFS —> not complete)
2. **Optimality** :
   - An Algorithm is Optimal if it is guaranteed to find the best solution (UCS)
3. **Time Complexity** :
   - How long an algorithm takes to find a solution typically in terms of the number of nodes generated during the search .
   - measured by the number of nodes generated during the search.
4. **Space Complexity**:
   - How much memory an algo needs to perform the search .
   - measured by the maximum number of nodes stored in memory at any point during the search .

Search Complexity :

- Complexity in search graph is expressed using the following :
  Branching factor(b) : the maximum number of successors (children) of any node in the graph .
  Depth of the shallowest Goal (d) : The number of steps from root to the shallowest goal node .
  Maximum path length(m) : The length of the longest path in the state space .

Search cost and total cost

1. **Search Cost** : the computational effort required to find a solution , it depends on (time complexity , memory usage)
2. **Total Cost** : combines the **search cost** and the **path cost** of the solution found
   - *Example* : From Arad to Bucharest , the search cost is the time taken by the search algo and the path cost is the total length of that path in km . and the total cost is the combination of the two .

Uninformed Search Strategies (Blind Search)

- These search strategies do not have any additional information about states beyond what is provided in the problem definition (only initial state , actions , goal)

> "*being in a maze with no map. You don't know which paths are better or closer to the exit—you just explore all possible paths step by step."

1. Breadth First Search :

- starts from the root node and expands all its successors then expand all those nodes and so on .
- Uses a **FIFO** queue for the frontier shallower nodes are expanded first , then we go for the deep ones .

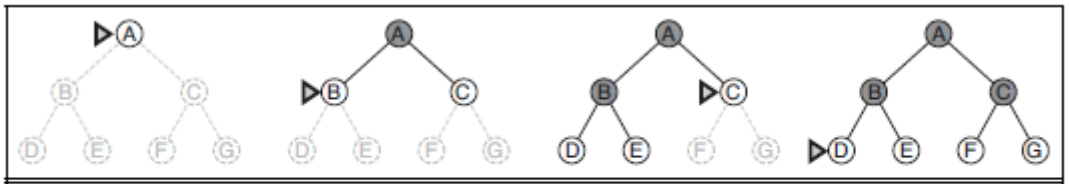
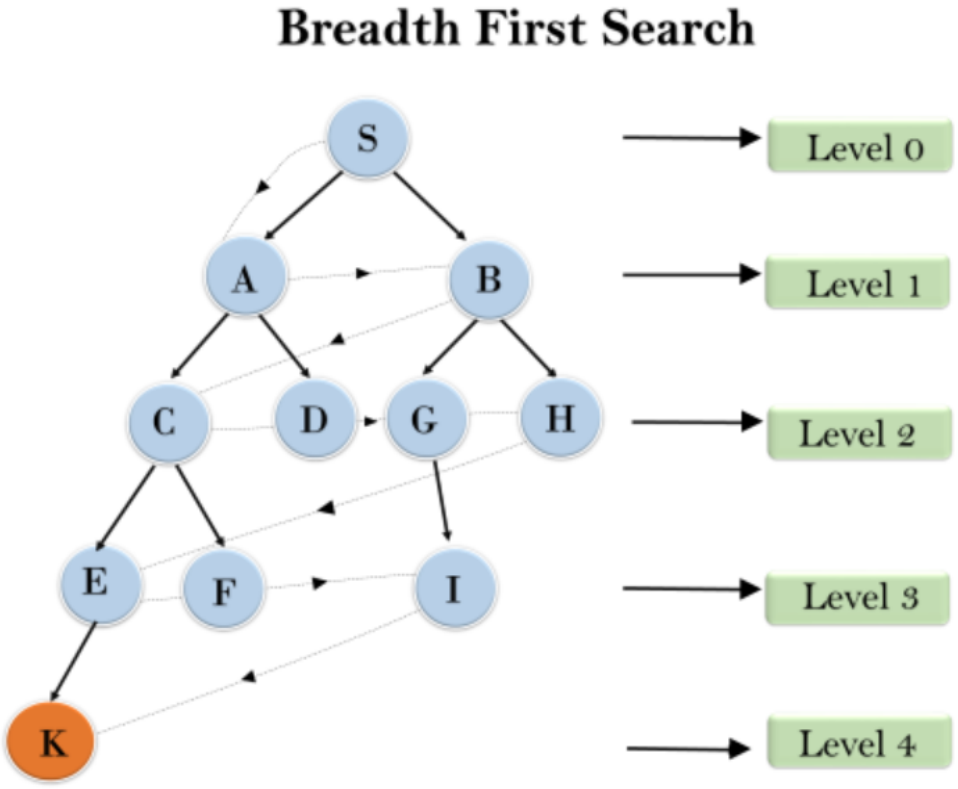


Figure 3.12 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by a marker.

- **Goal Test** : as soon as a node is generated BFS is applied , which ensures the the shallowest goal node is found as soon as it is generated .

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)  /* chooses the shallowest node in frontier */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier ← INSERT(child, frontier)
```

Figure 3.11 Breadth-first search on a graph.



Completeness	Optimality	Time Complexity	Space Complexity
BFS is **complete** . The goal node exists at a finite depth d , then BFS will eventually find it .	BFS is **optimal** , only if all actions have the same cost (the cost of reaching the goal doesn't decrease as we go deeper into search tree)	$O(b^d)$ max of sucessros(b) times itself (d times which is the depth , because the worst case if finding the goal at the deepest right-most node) $T(b) = 1+b^2+b^3+\ldots\ldots+b^d = O(b^d)$	- $O(b^d)$ BFS stores all nodes in the current depth level

Completeness	Optimality	Time Complexity	Space Complexity
		It could be at level 1 , 2 ... or d	

Uniform cost Search

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution, or failure

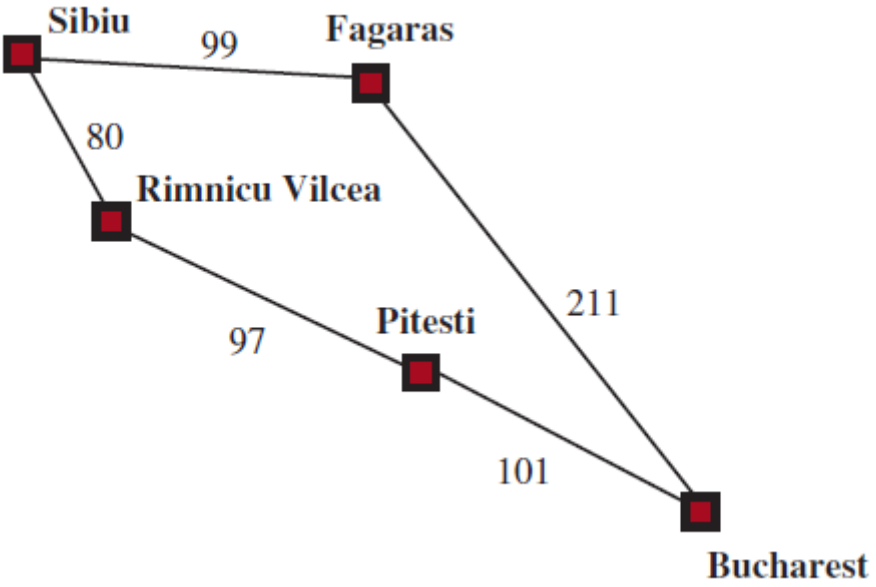
node ← a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0
frontier ← a priority queue ordered by PATH-COST, with *node* as the only element
explored ← an empty set
loop do
  if EMPTY?(*frontier*) **then return** failure
  node ← POP(*frontier*) /* chooses the lowest-cost node in *frontier* */
  if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)
  add *node*.STATE to *explored*
  for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**
    child ← CHILD-NODE(*problem*, *node*, *action*)
    if *child*.STATE is not in *explored* or *frontier* **then**
      frontier ← INSERT(*child*, *frontier*)
    else if *child*.STATE is in *frontier* with higher PATH-COST **then**
      replace that *frontier* node with *child*

Figure 3.14 Uniform-cost search on a graph. The algorithm is identical to the general graph search algorithm in Figure 3.7, except for the use of a priority queue and the addition of an extra check in case a shorter path to a frontier state is discovered. The data structure for *frontier* needs to support efficient membership testing, so it should combine the capabilities of a priority queue and a hash table.

- Uninformed search Algorithm , it expands the node with the **lowest path cost** first . it is optimal and complete

How UCS works ?

- **Priority Queue** : uses it to store nodes , the priority of a node is determined by its *path cost* $g(n)$ -the total cost from the start node to the current node n-
- it expands the node with the lowest path cost from the **frontier** .
- unlike BFS , the **Goal Test** is applied when a node it selected for expansion , not when it first generated
- **Explored set** : has the set of nodes that have already been expanded
- **Frontier Update** : if a better path is found to a node already in the frontier , the node is updated with the new path cost .



- we always choose the mpin cumultative path cost !
  from root create frontier list , and pop based on priority each time

Completeness	Optimality	Time Complexity	Space Complexity
ucs is complete	**UCS** is *optimal* for any step-cost function (as long as costs are non-negative).	**Time Complexity**: $O(b^{1+\lfloor C^*/\epsilon \rfloor})$, where: ◦ b = branching factor, ◦ C^* = cost of the optimal solution, ◦ ϵ = minimum step cost.	

Depth First Search :

- DFS expands the deepest node in the frontier first, effectively exploring one branch of the search tree as deeply as possible before backtracking to explore other branches.
- each time a node is expanded , it is removed from the frontier

1. Exploration:

- DFS starts at the root of the tree and explores one branch as deeply as possible.
- When it reaches a node with no successors (a leaf node), it stops and "backs up" to the next deepest node in the frontier that still has unexplored successors.
- This process continues until a solution is found or the entire tree is explored.

2. Data structure for frontier

- DFS uses a **LIFO (Last-In, First-Out) queue**, meaning the most recently generated node is expanded first. This ensures that the search always goes deeper into the tree.

3. Recursive Implementation:

- DFS can also be implemented recursively, where the algorithm calls itself on each child node in turn. This is a natural way to explore one branch at a time.

Pros	Cons
Memory efficient: stores only the current path	Not cost-optimal: may find a suboptimal solution
Low space complexity: O(bm) for tree search	Can get stuck in infinite loops in cyclic spaces
Explores deep branches quickly	Incomplete in infinite state spaces
No need for a "reached" table in tree search	Redundant paths: may revisit states multiple times
Useful for problems with deep solutions	Requires cycle-checking to avoid infinite loops
Works well for tree-like state spaces	Not systematic in infinite spaces
Backtracking variant saves even more memory	May explore unnecessary deep branches

Completeness	Optimality	Time Complexity	Space Complexity
DFS is **not complete** for infinite-depth or loops, but **complete** for finite state spaces.	DFS is **not optimal**—it may find a suboptimal solution if there are lower-cost solutions at shallower depths.	$O(b^m)$ where b is the branching factor and m is the maximum depth (which can be much larger than d, the depth of the shallowest solution).	$O(bm)$ for tree search $O(m)$ for backtracking search.

Backtracking Search: A Concise Overview

What It Is

- A memory-efficient variant of **Depth-First Search (DFS)**.
- Builds solutions incrementally, backtracking when hitting a dead end.

How It Works

- Generates **one successor at a time** instead of all at once.
- Modifies the **current state directly**, undoing changes when backtracking.
- Uses **O(m) memory**, where (m) is the max search depth.

Key Features

- Avoids infinite loops by checking for cycles on the current path.
- Systematically explores all possible solutions.
- Ideal for **puzzles, constraint satisfaction, and combinatorial optimization**.

Example: **N-Queens Problem**

- Places queens **row by row**, backtracking on conflicts.
- Finds a valid solution **without storing all possible states**.

Pros & Cons

✅ **Advantages**

- Memory-efficient.
- Systematic and flexible for various problems.

❌ **Disadvantages**

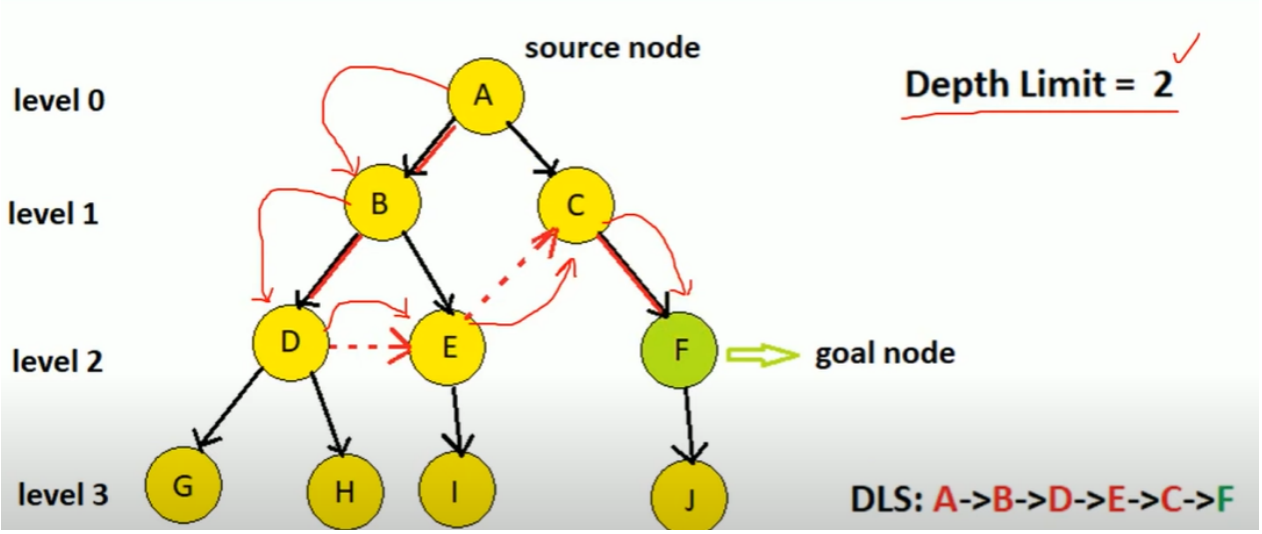
- Can be slow in worst-case scenarios.
- Requires **reversibility** (undoing actions).

Use Cases

- **N-Queens**, **Sudoku**, **scheduling**, **logic programming**.

4. Depth Limited Search

- DLS is an alternative for DFS since it fails as searching in infinite state spaces , DLS uses a **pre-determined** depth *limit* `l` where we can't go over this depth .
- DFS is DLS with l = ♾



⭕ Problem :

- if we choose $l < d$ and the goal is present at the depth d , then it's **incomplete**
- even if we choose $l > d$ DLS is **non-optimal**

🕛 Time complexity :

- $O(b^l)$ searches b^l nodes rather than b^m

📰 Space complexity ;

- $O(bl)$

Diameter of the State Space:

- The **diameter** is the maximum number of steps (or depth) required to go from any state to any other state in the state space.
- For example, in a map of cities, if the diameter is 9, it means you can travel from any city to any other city in at most 9 steps.
- Knowing the diameter helps set a **depth limit** for the search, making it more efficient.

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
      cutoff_occurred? ← false
      for each action in problem.ACTIONS(node.STATE) do
          child ← CHILD-NODE(problem, node, action)
          result ← RECURSIVE-DLS(child, problem, limit − 1)
          if result = cutoff then cutoff_occurred? ← true
          else if result ≠ failure then return result
      if cutoff_occurred? then return cutoff else return failure
```

Figure 3.17 A recursive implementation of depth-limited tree search.

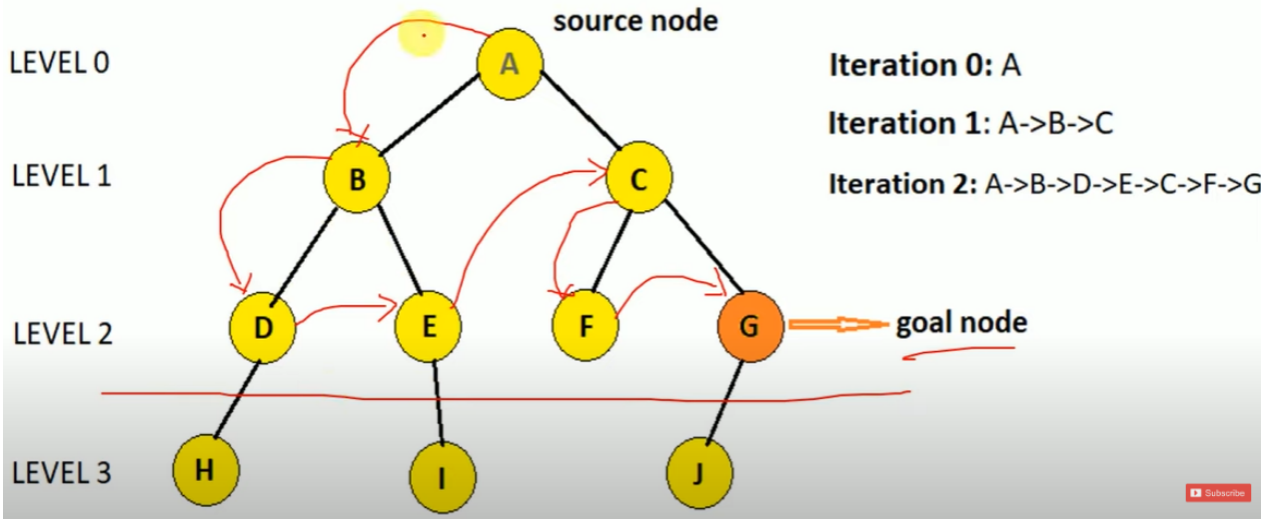
- the standard *failure* value indicates no solution; the *cutoff* value indicates no solution within the depth limit

5. Iterative deepining Search

- used in combination with dfs , it finds the **best depth limit**

How it works ?

- it gradually increases the limit (0 → 1 → 2 ...) until a goal is found, meaning when the depth limot reaches d *shallowest goal node depth*



```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
    for depth = 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH(problem, depth)
        if result ≠ cutoff then return result
```

Figure 3.18 The iterative deepening search algorithm, which repeatedly applies depth-limited search with increasing limits. It terminates when a solution is found or if the depth-limited search returns *failure*, meaning that no solution exists.

Title

It combines the benefits of BFS and DFS

- like DFS its memory requirments are modest $O(bd)$
- like BFS , it's complete when b is finite and optimal when path cost is non-decreasing function of the depth of the node.

Complexities

IDS might seem wasteful since we generate the states multiple times but it's not as much (bottom level → once , next to bottom → twice .. until children of root whoch are generated d times)so the total of nodes generated is :

$$N(\text{IDS}) = (d)b + (d-1)b^2 + \cdots + (1)b^d \ ,$$

🕐 **Time complexity** : is $O(b^d)$ ~ same as one of BFS (there is additional cost for generating the upper levels multiple times but we can neglect it)
📰 **Space Complexity** : is $O(bd)$

Hybrid approach

- Run **BFS** until memory is almost full.
- Then switch to **iterative deepening** from all nodes in the frontier (the nodes at the current depth level).
- This combines the memory efficiency of iterative deepening with the completeness of BFS.

6. Bidirectional Search

- we run two simultaneous searches , one **forward from the initial state** and the other **backward from the goal state** , hoping that the two will meet in the middle (solution was found)

How it works ?

- we start two searches from goal and initial state , and we replace the goal test by a check to see if the frontiers of two searches **intersect** (the possible nodes to expand)

then solution is found

👀 **How do we search backward ?**

- The backward search requires computing **predecessors** (states that lead to the current state).
- Predecessors are easy to compute if actions are reversible; otherwise, it requires ingenuity.

🥅 **What if there is multiple goal states ?**

- If there are multiple goal states, a **dummy goal state** can be created with all actual goals as its predecessors.
- For abstract goals (e.g., "no queen attacks another"), bidirectional search is harder to implement.

Complexities

🕐 Time Complexity

- $O(b^{d/2})$

📰 **Space Complexity**:

- $O(b^{d/2})$, as both frontiers must be stored in memory.
  - Can be reduced slightly by using iterative deepening for one search, but one frontier must still be kept in memory.

Property	Description
Completeness	**Yes**, if there is only a single goal; otherwise, it can be difficult.
Time Complexity	$O(b^{d/2})$
Space Complexity	$O(b^{d/2})$ (Higher space requirement).
Optimality	*Yes*, but**not always optimal**, even if both searches are BFS.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[a]	Yes[a,b]	No	No	Yes[a]	Yes[a,d]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes[c]	Yes	No	No	Yes[c]	Yes[c,d]

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: [a] complete if b is finite; [b] complete if step costs $\geq \epsilon$ for positive ϵ; [c] optimal if step costs are all identical; [d] if both directions use breadth-first search.

Informed (heuristic) Search Strategies

- **Informed search** , uses problem specific knowledge beyond the definition of the problem itself , it can find solution more efficiently than uninformed search .
- **Best first Search** : an instance of GENERGAL TREE SEARCH or GRAPH SEARCH algo where the node selected for the expansion based on an **evaluation function f(n)** , taken

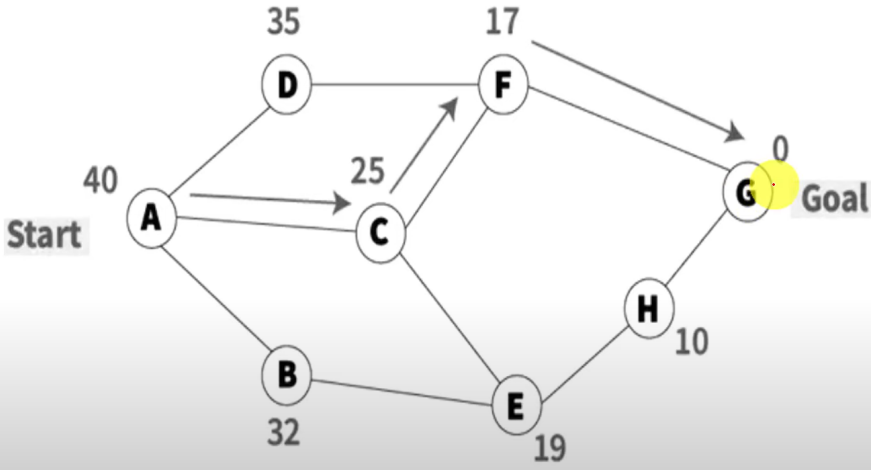
as *cost estimate* so the node with the lowest evaluation is expanded first .
    - it is impelmented as UCS except for g → f for the order of the priority queue .

The choice of f :

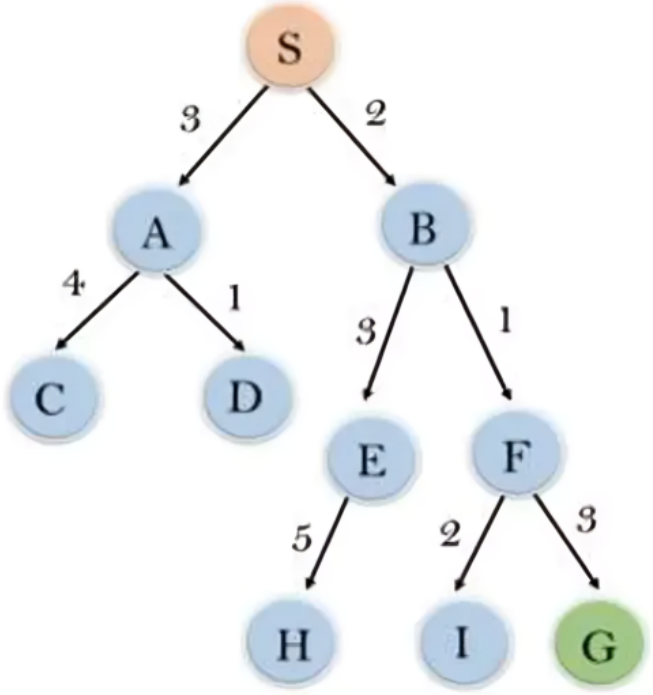
- it determines the search strategy , and most of em include as a component of f a heuristic function $h(n)$
    - $h(n)$: estimated cost of the *cheapest path* from state at node n to the goal state .
        - they are arbitray , non-negative , problem specific functions with one constraint *h(n=goal) =0*

1. Greedy best-first Search :

- it expands the node that is closest to the goal meaning the one with the lowest $f(n)$ value .
    - **f(D) = h(D) = 35**
    - **f(G) = h(G) = 0**



- greedy best first search ignores the cost of the path that has already been traversed to reach n , which makes it sometimes **not necessarily optimal**



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

S → B → F → G is the path which costs 8

✅ Advantages

- **Fast**: Expands fewer nodes if the heuristic is good.
- **Low Search Cost**: Minimizes exploration of non-solution paths.

❗ Disadvantages

- **Not Optimal**: May find ***suboptimal*** solutions (e.g., longer paths).
- **Incomplete**: Can get stuck in infinite loops or dead ends in finite state spaces (tree search version).
- **Heuristic Dependency**: Performance relies heavily on the quality of the heuristic.

Complexity

- **Worst-case**: $O(b^m)$
- b is the branching factor and m is the maximum depth.
- **With a Good Heuristic**: Complexity can be significantly reduced.

Completeness	Optimality	Time Complexity	Space Complexity
Incomplete without systematic checking of repeated states.	**Not optimal**, may find suboptimal solutions.	**Worst case:** $O(b^m)$, where b is the branching factor and m is the maximum path length.	**Worst case:** $O(b^m)$, requiring storage for explored nodes.

A* Search :

- in A* search we evalute the node based on $f(n) = g(n) + h(n)$
  where :
- $g(n)$ is the cost to reach the node .
- $h(n)$ the estimated cost to get from the node to the goal

Completeness and Optimality

- A* search is only complete and optimal under some certain conditions

1. For Optoimality

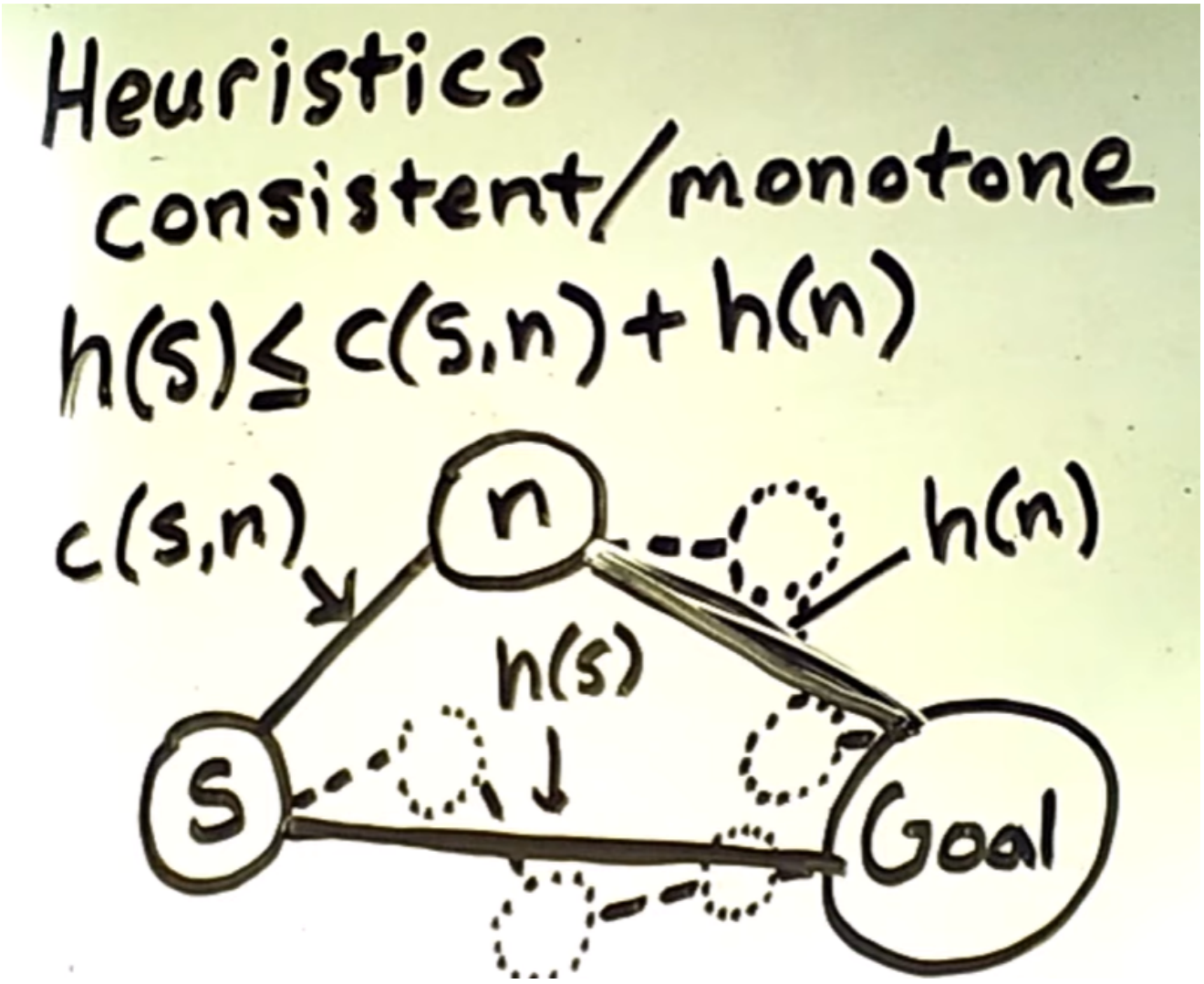
💥 **Admissibility** :
A heuristic $h(n)$ is **admissible** if it never overestimates the true cost to reach the goal:
$h(n) \leq h^*(n)$

- where $h^*(n)$ is the actual cost from node n to the goal.
- Admissible heuristics are **optimistic**, assuming a lower or equal cost than reality.
- Ensures **A*** search finds the optimal solution.
  Example:

- In route-finding problems, **straight-line distance (SLD)** is admissible since the shortest path between two points is a straight line.
- If the true cost is **100**, an admissible heuristic may estimate **80** but never **120**.
  Why is Admissibiloty important ?
- Guarantees **A*** search never overlooks the optimal solution.
- If $h(n)$ **overestimates**, A* may choose a **suboptimal** path.

consistency

Consistency (Monotonicity)



A heuristic $h(n)$ is **consistent** if it satisfies the **triangle inequality**:
$h(n) \leq c(n, a, n') + h(n')$
where:

- $c(n, a, n')$ is the cost to move from node n to its successor n' via action a.
- $h(n')$ is the heuristic estimate from n' to the goal.

Why is Consistency Important?

- Ensures the heuristic is **locally consistent**.
- Guarantees that the estimated cost from n to the goal is **never greater** than moving to a successor n' and then estimating from there.

- Ensures **A*** search never **revisits** nodes, improving efficiency.

proof that consistency implies admissibility : proof

> **Key Idea**
>
> A heuristic is **consistent** if, for every edge (n, n') with cost c(n, n'), it satisfies:
> $h(n) \leq c(n, n') + h(n')$

- If this inequality **fails for even one edge**, the heuristic is **inconsistent** (not monotonic).
- If the inequality **holds for all edges**, the heuristic is **consistent**.

🎄 **When is A* Optimal ?**

1. Tree search version of A *is* optimal* if h is admissible .
2. Graph search version is *optimal* if h is consistent

Cost :

if $C*$ is the cost of the optimal solution path then A* does the following :

- **expand nodes with $f(n) < C*$:**
  - their total cost is less than optimal cost (admissability of heuristic function)
- **may expand nodes with $f(n) = C*$**
  - before selecting the goal node (they lie on the optimal path)
- **does NOT epand nodes with $f(n) > C*$:**
  - A* *prunes* (ignores) nodes that have a f(n) value greatrer than the cost of the optimal path

2. For Completeness

A* is complete only if :

- All step costs exceed some finite ϵ (don't have a zero cost or else that will leed to infinet loop) && The branching factor b is finite (to avoid infinite state space) .

Optimal Efficiency

- A *is optimally efficient for* any given consistent heuristic , *meaning that no other optimal algorithm is guaranteed to expand fewer nodes than A* .

Limitations of A*

🕐 **Time Complexity** : it can expand an exponential number of nodes in the worst case

- The time complexity depends on the heuristic error:
- **Absolute error:** $\Delta = h^* - h$, where h^* is the true cost.
- **Relative error:** $\epsilon = \frac{(h^* - h)}{h^*}$.
- For constant step costs, the time complexity is $O(b^{\epsilon d})$, where d is the solution depth.

📰 **Space Complexity** : A* stores all generated nodes in memory , which can lead to

memory exhaustion

- space complexity if implemented using tree search $O(b^m)$
- space complexity if implemented using graph search $O(b^d)$

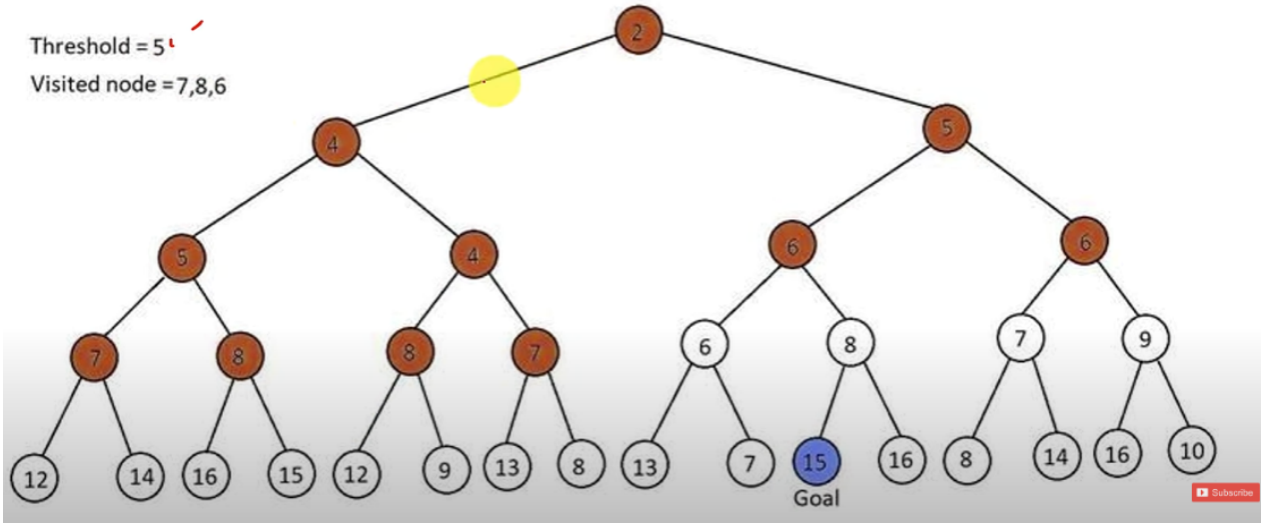
Practical Workarounds

- **Suboptimal Variants:**
  Use variants of A *that find suboptimal solutions quickly (e.g., weighted A*).
  These algorithms trade *optimality* for *speed* and *memory* efficiency.
- **Non-Admissible Heuristics:**
  Design *heuristics* that are *more accurate* but <u>not strictly admissible.</u>
  These *heuristics* may ***overestimate the cost in some cases but can still provide good solutions.***
- A* as any Graph search Algorithm stores all generated nodes in memory which makes it run our of space before time.

3. Memory bounded heuristic Search

1. Iterative-deepening A *(IDA)* :

- same idea as id search , but we set the limit (threshlod) to be based on the cost function ($f(n) = g(n) + h(n)$)
- it helps *Reducing the memory size* requirments for A*



Steps :

1. **Set root as current node** and calculate its f-score
2. **Set threshold** : set the first threshold to be the root's f-score
3. **Expand nodes** only if f(node) < threshold
4. **prune other nodes** and put them in visited nodes list
5. **return goal** in case on of the expanded nodes is the goal state
6. **Update threshold** to the min of the visited nodes list and redo steps all over again , starting from the root !

Advantages:

- **Memory-efficient**: Only stores the current path, so space complexity is $O(d)$, where d is the solution depth.
- **No priority queue**: Avoids the overhead of maintaining a sorted queue of nodes.

Limitations:

- **Inefficient for real-valued costs**: If step costs are not uniform, IDA* can perform many redundant expansions.
- **Higher time complexity**: Can be worse than A* due to repeated node expansions.

2. Recursive best first search (RBFS)

- similar to recursive depth first search 'uses only **linear space**'
  - it uses *f-limit* variable to keep track of the f-value of the best alternative path available from any ancestor of the current node .
  - if current node exceeds this limit we rewind back to the alternative path .
  - as we go back the RBFS replaces *f-value* of each node along the path with *backed up value* (which is the best f-value of it's children)

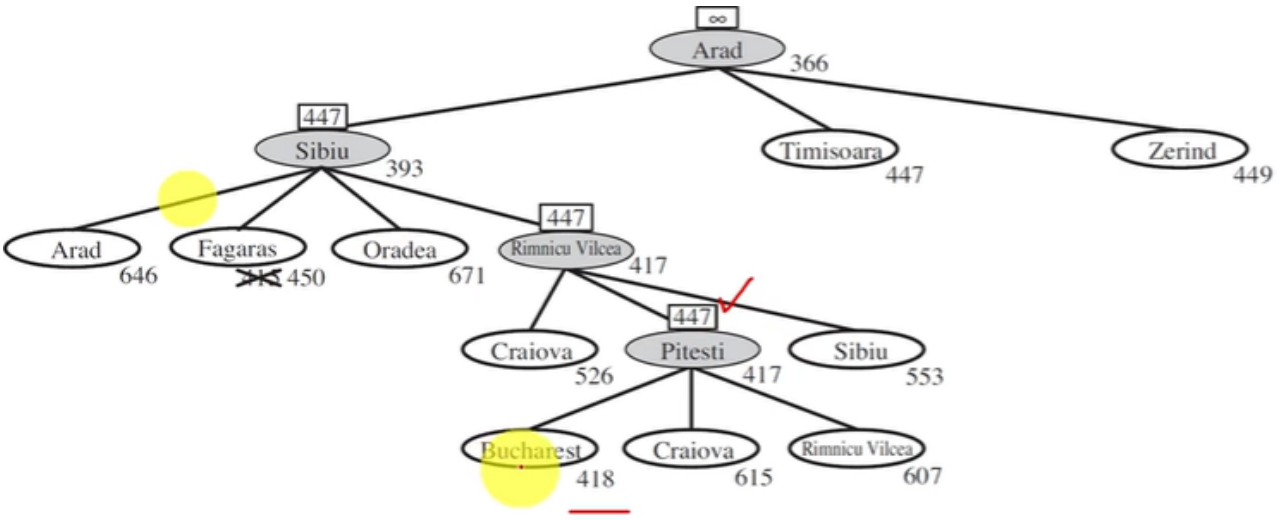
$$s.f = max(s.g + s.h, node.f)$$

$$Best = best\ successor\ node$$

$$Best.f > f_limit\ return\ failure$$

$$Alt = second\ best\ succ\ node$$

$$f_limit = min(f_limit, alt.f)$$



we calculate $f(node) = max(f(node) + f(parentnode)$) then we set the children's f-values , among them we choose the best successor node , then we check if that value if greater then the *f-limit* which we initially set as ♾ , if not we choose and alternative which is the second best sucessor . we then update the *f-limit* to $min(f_limit, f - value(altnode))$

```
function RECURSIVE-BEST-FIRST-SEARCH(problem) returns a solution, or failure
    return RBFS(problem, MAKE-NODE(problem.INITIAL-STATE), ∞)

function RBFS(problem, node, f_limit) returns a solution, or failure and a new f-cost limit
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  successors ← [ ]
  for each action in problem.ACTIONS(node.STATE) do
      add CHILD-NODE(problem, node, action) into successors
  if successors is empty then return failure, ∞
  for each s in successors do /* update f with value from previous search, if any */
      s.f ← max(s.g + s.h, node.f))
  loop do
      best ← the lowest f-value node in successors
      if best.f > f_limit then return failure, best.f
      alternative ← the second-lowest f-value among successors
      result, best.f ← RBFS(problem, best, min(f_limit, alternative))
      if result ≠ failure then return result
```

Figure 3.26 The algorithm for recursive best-first search.

🕐 **Time Complexity** : $O(b^d)$
📰 **Space Complexity** : O(d) it's memory effcient and no need for priprity queue

Limitations :

- **Redundant work** : can re-expand nodes multiple times , increasing time complexity .

Completness	Optimality	Space Complexity	Time Complexity
Complete only if step cost is non-zero and branching facor is not infinte	*optimal* if h(n) is admissible	linear in the depth of the deepest optimal solution $O(d)$	potentially exponential but hard to set , since it depends on the accuracy of h(n) and on hwo oftern the best path cahnges as nodes are expanded

there are two additional algos in the book but were not mentioned in the slides MA *and SMA* , you can check them out ig ?

Learning to Search Better

Instead of using fixed search strategies, an **agent can learn** how to search more efficiently by analyzing its past search experiences and avoiding unpromising paths. This is done by

exploring the **metalevel state space** (the algorithm's internal state) and optimizing the trade-off between computational cost and solution quality.

Heuristic functions

Heuristics for the 8-Puzzle

1. The 8-Puzzle heuristic

- The **8-puzzle** consists of a 3×3 grid with 8 tiles and one empty space.
- Tiles can slide **horizontally or vertically** into the empty space.
- The goal is to rearrange the tiles from a **start state** to a **goal state**

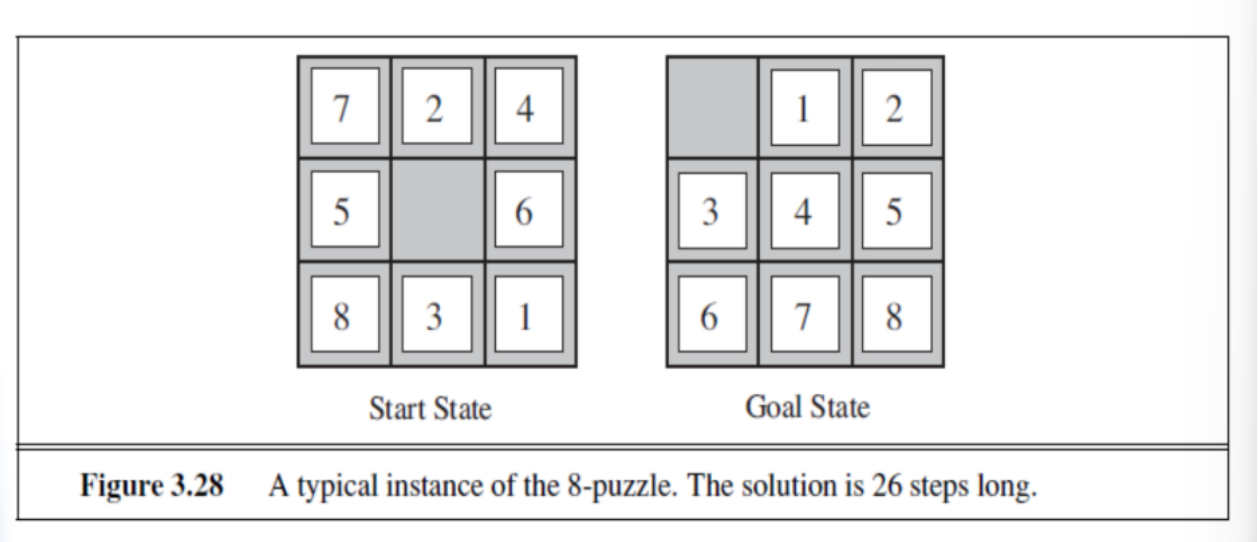


Figure 3.28 A typical instance of the 8-puzzle. The solution is 26 steps long.

- The **average solution cost** for a random 8-puzzle is about *22 steps*.
- The **branching factor** (number of possible moves) is about **3**:
  - **4 moves** if the empty tile is in the center.
  - **2 moves** if the empty tile is in a corner.
  - **3 moves** if the empty tile is on an edge.
- Without heuristics:
  - An **exhaustive tree search** to depth 22 would explore about $3^{22} \approx 3.1 \times 10^{10}$ states.
  - A **graph search** reduces this to **181,440 distinct states** (since only $9!/2 = 181,440$ states are reachable).
- For larger puzzles like the **15-puzzle**, the number of states explodes to 10^{13}, making heuristics essential.

🦩 What makes a good heuristic ?

- A good heuristic for A* must be **admissible**: it should **never overestimate** the true cost to reach the goal.
- Two commonly used heuristics for the 8-puzzle are:

a) h_1: Number of Misplaced Tiles

- **Definition**: Counts the number of tiles that are not in their goal position.
- **Example**: In the start state of Figure 3.28, all 8 tiles are misplaced, so $h_1 = 8$.
- **Why Admissible?**:

- Each misplaced tile must move at least once to reach its goal position. (kichghol we assume belli h1 returns how many moves we need assuming we only move by one to reach goal state)
- Therefore, h_1 never overestimates the true cost.

b) h_2: Manhattan Distance

- **Definition**: Sum of the horizontal and vertical distances of each tile from its goal position.
  - Also called the **city block distance**.
- **Example**: For the start state in Figure 3.28:
  - Tile 1: 3 steps away.
  - Tile 2: 1 step away.
  - Tile 3: 2 steps away.
  - Tile 4: 2 steps away.
  - Tile 5: 2 steps away.
  - Tile 6: 3 steps away.
  - Tile 7: 3 steps away.
  - Tile 8: 2 steps away.
  - Total: $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$.
- **Why Admissible?**:
  - Each move can only move one tile one step closer to its goal (and we don't take into consideration moving the other tilmes that can be required to reach the goal .
  - Therefore, h_2 never overestimates the true cost.
- h_2 is **more informed** than h_1 because it considers the distance of each tile from its goal, not just whether it's misplaced.
  - As a result, h_2 usually leads to **fewer node expansions** and faster search.

2. Affect of heuristic accuracy on performance

The Effect of Heuristic Accuracy on Performance

🧠Effective Branching factor

- The **effective branching factor** is a way to measure the quality of a heuristic.
- It represents the ***average number of successors*** (branches) per node that A* would generate if the search tree were uniform. (kol node has a fixed number of branches)
- Mathematically, if N is the total number of nodes generated by A* and d is the solution depth, then:

$$N + 1 = 1 + b^* + (b^*)^2 + \cdots + (b^*)^d \, .$$

- A lower $b*$ means the heuristic is more **effective**, as it reduces the number of nodes A *needs to explore.*
  👻 *Why is $b^$* Important?
- b^* helps us understand how efficient a heuristic is:
  - A well-designed heuristic will have $b*$ close to 1, meaning the search tree is **narrow** and A* explores fewer nodes.

- A poor heuristic will have a higher b^*, leading to a **wider** search tree and more nodes being explored.

Important on heuristic dominance:

🤦‍♂️best heuristic in textbook example

- If one heuristic **dominates** another (e.g., h2 dominates h1), it will always perform at least as well, and usually better.

3. Best heuristic :

- if we can't find a *clearly best* heuristic , then we take the max and it is clearly still admissible and can be proven to be consistent !
- in the 8 puzzle problem ,we have an average of 3 for the branching factor
- using tree search (exhaustive) ⟶ too much space
- using graph search ⟶ less space but still bad
- h1 : how many tiles are in place (admissible)
- h2 : the sum of the distances of the tiles from their goal position(admissible)
- **effective branching factor** : helps characterizing the quality of the heuristic

🖉 Midterm qst Impo :

- **Heuristic Domination**: If $h_2(n) \geq h_1(n)$ for all n, h_2 dominates h_1.
- **Admissibility**: $h(n) \leq h^*(n)$ (never overestimates true cost).
- **Efficiency**: A* with h_2 expands fewer nodes than with h_1 if h_2 dominates and is admissible.
- h_2 **is Better**:
- If h_2 dominates h_1 **and** both are admissible.
- h_2 **is Not Better**:
- If h_2 is not admissible (e.g., $h_2(n) = 2 \cdot h_1(n)$). - **Constructing** h_2:
- Requires problem-specific knowledge.
- Hard to ensure both domination and admissibility.
- **Conclusion**: h_2 is better only if it dominates h_1 and remains admissible. Otherwise, h_1 is safer for optimality.

4. Learning heuristics from Experience

Manually designing heuristics can be time consuming an it might not always yield the best results , but instead we can learn heuristics automatically by analyzing examples of solved problems

Steps

- solve many instances of the problem and record the state at each step and the actual cost
- we define **features** of the state that are relavent to predicting the solution cost (*Example* : $x1(n)$ ⟶ number of misplaced tiles and $x2(n)$ → number of pairs of adjacent tiles that are not adjacent in the goal state)

- Learn the heuristic using a **learning algorithm** (neural networks , decision trees ..) to find the relationship between the features and the actual solution's cost (*example* : inductive learning $h(n) = c1x1(n) + c2x2(n)$)

Algorithm	Description	Optimal?	Complete?	Use of Heuristic?	Efficiency	W Us
Breadth-First Search (BFS)	Expands the shallowest nodes first.	Yes (if all costs are equal)	Yes	No	High time and space complexity ((O(b^d))), where (b) is branching factor and (d) is depth.	W go cl th an br fa sn
Depth-First Search (DFS)	Expands the deepest nodes first.	No	No (in infinite spaces)	No	Low space complexity ((O(bm))), but can get stuck in infinite loops.	W m lin an go de th sp
Uniform-Cost Search (UCS)	Expands the node with the lowest path cost (g(n)).	Yes	Yes	No	Expands all nodes with (g(n) < C^ *), where (C^*)* is the optimal cost.	W ac co ec wh he av
Greedy Best-First Search	Expands the node with the lowest heuristic cost (h(n)).	No	No (can get stuck in loops)	Yes	Can be very fast but may not find the optimal solution.	W go he av an op is re
A*	Expands the node with the lowest (f(n) = g(n) + h(n)).	Yes (if (h(n)) is admissible)	Yes	Yes	Expands fewer nodes than UCS due to the heuristic.	W go he av an op is re

Algorithm	Description	Optimal?	Complete?	Use of Heuristic?	Efficiency	W Us
Iterative Deepening DFS (IDDFS)	Combines DFS and BFS by iteratively increasing the depth limit.	Yes (if all costs are equal)	Yes	No	Time complexity similar to BFS, but space complexity of DFS.	W m lin an go is un
Bidirectional Search	Runs two simultaneous searches: one from the start and one from the goal.	Yes	Yes	No	Reduces the search space significantly ((O(b^{d/2}))).	W go is an br fa hi
Depth-Limited Search (DLS)	DFS with a depth limit.	No	No (if the goal is beyond the limit)	No	Avoids infinite loops but may miss the goal if the limit is too small.	W se sp lar th de kn
Iterative Deepening A *(IDA)*	Combines A* and IDDFS by iteratively increasing the (f(n)) limit.	Yes (if (h(n)) is admissible)	Yes	Yes	Space-efficient version of A*.	W m lin an go he av
Hill Climbing	Local search algorithm that moves to the best neighboring state.	No	No	Yes	Can get stuck in local optima.	W se sp to fo sy se
Simulated Annealing	Probabilistic local search that allows worse moves to escape local optima.	No	No	Yes	Better at escaping local optima than Hill Climbing.	W se sp ma op
Genetic Algorithms (GA)	Population-based search inspired by natural selection.	No	No	Yes	Good for large, complex search spaces with no clear structure.	W pr co an tra se m

Algorithm	Description	Optimal?	Complete?	Use of Heuristic?	Efficiency	W Us
						ar in